Guía de estudio del Frontend
Proyecto ISCGB – AutoGestión

Programación III · Segundo parcial
Ejes 1, 2 y 3: Frontend (TypeScript y componentes, routing y formularios, servicios y comunicación con la API)

> **Cómo usar esta guía**
> Cada tema del examen tiene tres partes: **el concepto** (lo que te pueden preguntar en general), **cómo está en nuestro proyecto** (con el archivo y el código real) y, cuando el tema **no está en el proyecto**, una explicación para responderlo igual, apoyándose en lo visto durante el año. Al final hay preguntas tipo examen con respuestas cortas.

> **Alcance**
> Solo el frontend (`ISCGB_Frontend` , SPA en Angular 21). Los ejes 4, 5 y 6 son del backend y están en la guía del backend; acá solo se marca dónde se tocan con la API.

Etiquetas usadas: EN EL PROYECTO está implementado · PARCIAL está en parte o de otra forma · NO ESTÁ hay que responderlo con la teoría.

Índice

1. **Panorama del proyecto**: tecnologías, carpetas, recorrido de un pedido y mapa de pantallas.
2. **Eje 1 – TypeScript y componentes**
   - 1.1 SPA y Angular como framework
   - 1.2 TypeScript: tipos, interfaces y clases
   - 1.3 Componentes, decoradores y Angular CLI
   - 1.4 Templates y data binding
   - 1.5 Comunicación entre componentes: input() y output()
   - 1.6 Directivas
3. **Eje 2 – Routing y formularios**
   - 2.1 Rutas y router-outlet
   - 2.2 Navegación: routerLink y Router
   - 2.3 Parámetros de ruta y query params
   - 2.4 Carga diferida (lazy loading)
   - 2.5 Guards
   - 2.6 Formularios reactivos y validaciones
4. **Eje 3 – Servicios, comunicación con la API y estado**
   - 3.1 Servicios e inyección de dependencias
   - 3.2 Servicios con mock: clase abstracta y useClass
   - 3.3 HttpClient y Observables
   - 3.4 Interceptores
   - 3.5 Signals: signal, computed y effect
   - 3.6 Sesión y JWT del lado del front
5. **Puntos de contacto con el backend** (ejes 4 a 6)
6. **Preguntas tipo examen**
7. **Glosario**

1. Panorama del proyecto

Qué es

El frontend es una **SPA** (*Single Page Application*) hecha en Angular: el navegador descarga la aplicación una vez y después solo le pide datos en JSON a la API. Cada rol (Director, Secretario, Docente y Alumno) tiene su panel y sus pantallas: legajos, justificativos, calendario de parciales, mesas, materias, programas, certificados, reconocimiento de saberes y contratos de alumnos.

Tecnologías

Tecnología	Para qué la usamos
Angular 21 (*standalone* y *zoneless*)	Framework de la SPA: componentes, router, formularios, HttpClient e inyección de dependencias.
TypeScript 5.9	JavaScript con tipos. Se transpila a JavaScript al compilar.
RxJS 7.8	Observables: lo que devuelve `HttpClient` y los flujos asincrónicos.
Signals (`@angular/core`)	El estado reactivo de las pantallas y de los servicios.
Tailwind CSS v4 + SCSS	Estilos con clases utilitarias. Los colores son variables CSS, lo que permite el tema claro y oscuro.
Vitest	Pruebas unitarias (`npm test`): más de 100 archivos `.spec.ts` .
Angular CLI (`@angular/build`)	Generar piezas (`ng g c`), servidor de desarrollo (`ng serve`) y build de producción.

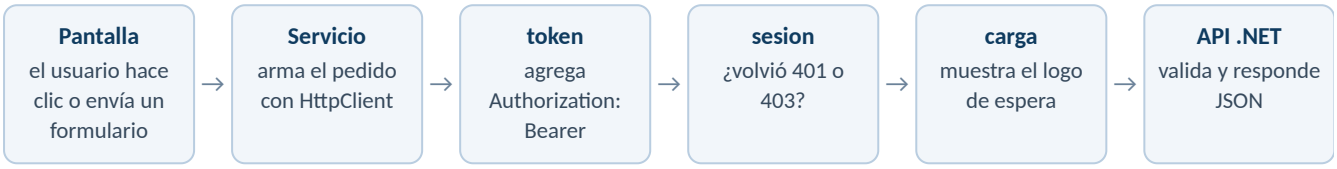
Organización de carpetas

Carpeta / archivo	Responsabilidad
`src/main.ts`	Punto de entrada: `bootstrapApplication(App, appConfig)` .
`app/app.config.ts`	Configuración global: router, HttpClient, interceptores, idioma `es-AR` y qué implementación usa cada servicio.
`app/app.routes.ts`	Las 36 rutas: qué componente se carga en cada URL y qué guards la protegen.
`app/core/`	Lógica sin pantalla, una carpeta por módulo (18): servicios, modelos (interfaces), guards, interceptores y funciones puras.
`core/configuracion/api.ts`	El único lugar con URLs de la API (`URL_BASE_API` y `RUTAS_API`).
`app/shared/ui/`	24 componentes reutilizables: `estructura-panel` , `insignia-estado` , `zona-archivo` , `dialogo-modal` …
`app/shared/directivas/`	Directivas reutilizables: `*siRol` .
`app/features/`	Las pantallas, agrupadas por rol o trámite (`director/` , `alumno/` , `legajo/` …). Lo que usa una sola pantalla va en su carpeta `partes/` .

Carpeta / archivo	Responsabilidad
`src/styles.scss`	Estilos globales y colores del sistema de diseño (`@theme`), incluido el tema oscuro.
`docs/`	Documentación del equipo: patrones obligatorios, contrato de la API y guía para pantallas nuevas.

Recorrido de un pedido

Todo pedido a la API recorre el mismo camino: **pantalla → servicio → interceptores → API**. El orden de los interceptores está en `app.config.ts` :




```
// app.config.ts — orden de los interceptores
provideHttpClient(
  withFetch(),
  withInterceptors([tokenInterceptor, sesionInterceptor, cargaInterceptor]),
),
```

La respuesta vuelve por el mismo camino: el servicio la convierte a un modelo del front y la pantalla la guarda en un *signal*, que redibuja la vista.

Mapa de pantallas (rutas)

Rutas	Quién entra	Qué hace
`login` , `recuperar-contrasena` , `crear-password` , `restablecer-password`	Público	Iniciar sesión, pedir el enlace de recupero y crear o restablecer la contraseña desde el mail.
`inicio` , `perfil` , `configuracion` , `justificativos/cargar`	Cualquiera con sesión	Inicio sin rol, "Mi perfil", configuración y carga de justificativos de inasistencia.
`director/panel` , `director/usuarios/nuevo` , `director/usuarios/:idUsuario/editar`	Director	Panel, alta y edición de usuarios.
`secretario/panel` , `secretario/listados` , `secretario/reconocimiento-saberes` , `secretario/frecuencia-avisos`	Secretario	Panel, listado de personas, solicitudes de reconocimiento y frecuencia de avisos.
`secretario/control-legajos` , `secretario/contratos` , `secretario/limite-parciales` , `mesas-examen` , `materias/asignaciones` , `suplencias` , `legajo/usuario/:idUsuario`	Director y Secretario	Control de legajos y legajo de cada persona, contratos de alumnos, límite de parciales, mesas, materias y suplencias.
`docente/panel` , `docente/entrega-programa` , `docente/mesas` , `docente/libro-de-temas`	Docente	Mi legajo, entrega del programa de la materia, mesas asignadas y libro de temas.

Rutas	Quién entra	Qué hace
`legajo/subir-documento`, `legajo/mis-documentos`	Docente y Alumno	Subir un PDF al legajo y ver el estado de cada documento.
`alumno/panel`, `alumno/certificado/regular`, `alumno/certificado/regular-con-horario`, `alumno/reconocimiento-saberes`	Alumno	Mi legajo, certificados en PDF y pedido de reconocimiento de saberes.
`calendario`	Docente, Secretario y Director	Calendario de parciales.
`sistema-diseno`, `''` y `'**'`	Público	Muestrario del sistema de diseño; la raíz redirige a `login`; cualquier otra URL muestra "Página no encontrada".

Lo que está siempre en pantalla

`app.html` tiene solo dos cosas: `<app-cargador-global />` (el logo de espera, que aparece en cualquier pedido HTTP) y `<router-outlet />`, donde el router dibuja la pantalla de la ruta activa. Al navegar cambia solo lo que está adentro del outlet.

2. Eje 1 – TypeScript y componentes

1.1 SPA y Angular como framework EN EL PROYECTO

Concepto. Una SPA carga una sola página HTML y cambia la vista sin recargar: el navegador baja la aplicación una vez y después solo pide datos. Antes, cada navegación traía una página completa desde el servidor. Angular es un **framework completo**, mantenido por Google: trae router, formularios, HttpClient, inyección de dependencias y la CLI, y TypeScript es obligatorio. React, en cambio, es una **librería**: resuelve la vista y el equipo elige el resto.

En el proyecto. `index.html` tiene un solo `<app-root>`; todo lo demás lo dibuja Angular. Las piezas oficiales salen del mismo framework: `@angular/router`, `@angular/forms` y `@angular/common/http` en `package.json`.

```
// src/main.ts — arranca la SPA con la configuración de app.config.ts
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

> **SSR: no lo usamos** NO ESTÁ
>
> La app se dibuja en el navegador (CSR). Con **SSR** el servidor ejecutaría Angular y mandaría el HTML ya armado: primera carga más rápida y mejor SEO. Para un sistema interno detrás de un login, el SEO no importa.

1.2 TypeScript: tipos, interfaces y clases EN EL PROYECTO

Concepto. TypeScript es JavaScript con **tipado estático**: todo JS válido es TS válido, pero el compilador revisa los tipos antes de transpilar. Los tipos **desaparecen** al transpilar: el navegador recibe JavaScript. Por eso `typescript` está en `devDependencies`: solo se usa para compilar.

Pieza	Qué es	En el proyecto
Tipos e inferencia	Se declaran (`let nombre: string`) o TS los deduce del valor inicial.	`signal(false)` infiere `boolean`; `signal<string | null>(null)` lo declara.
Interface	Contrato: la forma de un objeto. No genera JavaScript.	`Sesion`, `CredencialesLogin`, `RespuestaLogin` (`core/auth/modelos/`).
Clase	Datos y comportamiento. Existe en ejecución.	Todo componente y servicio: `export class Login`.
Clase abstracta	Declara qué métodos hay sin implementarlos.	`AuthService`, `LegajoService`, `ContratosService` …
Tipo de unión	Un valor que puede ser uno de varios tipos o literales.	`'Aprobado' | 'Pendiente' | 'Rechazado'`
Modificadores	`public`, `private`, `protected`, `readonly`.	`private readonly http = inject(HttpClient)`; `protected` = lo usa el template, no otras clases.
Genéricos	Un tipo que se pasa como parámetro.	`http.post<RespuestaLogin>(…)`, `Observable<Sesion>`.

```ts
// core/auth/modelos/credenciales-login.ts — lo que viaja en el login
export interface CredencialesLogin {
  dni: string;
  password: string;
}

// shared/ui/insignia-estado/insignia-estado.ts — unión + "type guard"
export type EstadoDocumento = 'Aprobado' | 'Pendiente' | 'Rechazado';

function esEstadoValido(valor: string | null): valor is EstadoDocumento {
  return valor === 'Aprobado' || valor === 'Pendiente' || valor === 'Rechazado';
}
```

Pregunta posible: ¿por qué `AuthService` es una clase abstracta y no una interface?

Porque las interfaces desaparecen al transpilar. Angular necesita algo que exista en ejecución para usarlo como "llave" de la inyección de dependencias (`provide: AuthService`). Una clase abstracta define el contrato igual que una interface, pero sí existe en el JavaScript final.

Los tipos no validan en ejecución

Si la API manda un campo de más o de menos, TypeScript no se entera: el chequeo fue al compilar. Por eso `ContratosHttpService` recibe la respuesta como `unknown` y la revisa campo por campo (`aContrato`) antes de convertirla en `ContratoAlumno` , e `insignia-estado` valida el estado con `esEstadoValido` , porque en la base es texto libre.

1.3 Componentes, decoradores y Angular CLI EN EL PROYECTO

Concepto. Un componente es un bloque de la interfaz con tres partes: la **clase** TypeScript (comportamiento), el **template** HTML (estructura) y los **estilos** (diseño). El decorador `@Component` le agrega a la clase la *metadata* que Angular necesita: selector, template, estilos e imports. Una pantalla es un árbol de componentes (patrón Composite).

En el proyecto. Todos los componentes son *standalone*: no hay `NgModule` , cada uno declara en `imports` lo que usa su template. Los archivos no llevan el sufijo `.component` (convención de Angular 20 en adelante): `login.ts` y `login.html` , con la clase `Login` .

```ts
// features/auth/login/login.ts
@Component({
  selector: 'app-login',                                  // la etiqueta <app-login>
  imports: [PanelBienvenida, FormularioLogin, SelectorTema], // lo que usa su template
  templateUrl: './login.html',
  changeDetection: ChangeDetectionStrategy.OnPush,        // se redibuja solo si algo cambió
})
export class Login {
  private readonly auth = inject(AuthService);
  private readonly router = inject(Router);
  protected readonly cargando = signal(false);
  protected readonly error = signal<string | null>(null);
  ...
}
```

Comando de la CLI	Qué hace
`npm install -g @angular/cli`	Instala la CLI global y deja disponible el comando `ng`.
`ng new nombre-del-proyecto`	Crea un proyecto (pregunta estilos y si habilitar SSR).
`ng serve` (`npm start`)	Servidor de desarrollo en `http://localhost:4200`.
`ng g c features/docente/libro-de-temas`	Crea un componente en esa carpeta (`g` = generate, `c` = component): `.ts`, `.html`, `.scss` y `.spec.ts`.
`--skip-tests`	No crea el `.spec.ts`.
`-t` y `-s`	Template y estilos dentro del `.ts` (*inline*), para componentes chicos.
`ng build`	Build de producción en `dist/`: JS transpilado, minificado y con *tree shaking*.
`ng test` (`npm test`)	Corre las pruebas con Vitest.

1.4 Templates y data binding EN EL PROYECTO

Concepto. El template conecta la vista con la clase. Hay cuatro formas de enlazarlas:

Sintaxis	Dirección	Ejemplo en el proyecto
`{{ valor }}` (interpolación)	clase → vista	`{{ apariencia().etiqueta }}` en `insignia-estado.html`
`[propiedad]="expr"` (*property binding*)	clase → vista	`[cargando]="cargando()"`, `[routerLink]="enlace.url"`, `[attr.aria-invalid]`
`(evento)="metodo()"` (*event binding*)	vista → clase	`(ngSubmit)="alEnviar()"`, `(enviar)="iniciarSesion($event)"`
`[formGroup]` + `formControlName`	ida y vuelta	El formulario de login (ver 2.6).

El **control de flujo** nuevo (`@if`, `@else`, `@for` con `track`) reemplaza a `*ngIf` y `*ngFor`. `track` le da a Angular la identidad de cada elemento, para reutilizar el DOM en vez de rehacer la lista: `@for (opcion of opciones(); track opcion.valor)` en `selector-tema.html`.

```
<!-- features/auth/login/login.html (sin las clases de estilo) -->
@if (sesionVencida() && error() === null) {
  <p role="status"><strong>Tu sesión venció.</strong> Por seguridad dura 2 horas…</p>
}
<app-formulario-login
  [cargando]="cargando()"
  [errorGeneral]="error()"
  (enviar)="iniciarSesion($event)"
/>
```

1.5 Comunicación entre componentes: input() y output() EN EL PROYECTO

Concepto. El padre le pasa datos al hijo (**entrada**) y el hijo le avisa al padre que pasó algo (**salida**, un evento). La forma clásica son los decoradores `@Input()` y `@Output()` con `EventEmitter`; la actual son las funciones `input()` y `output()`.

En el proyecto. Usamos el patrón **contenedor / presentacional**. El contenedor (`Login`) inyecta servicios y decide; el presentacional (`FormularioLogin`) solo recibe datos y emite eventos, no conoce ningún servicio, así que se reutiliza y se prueba solo. Los presentacionales de una pantalla viven en su carpeta `partes/` .

```
// features/auth/login/partes/formulario-login/formulario-login.ts
export class FormularioLogin {
  readonly cargando = input<boolean>(false);            // entrada: el padre avisa si está
esperando
  readonly errorGeneral = input<string | null>(null);   // entrada: el mensaje de error del login
  readonly enviar = output<CredencialesLogin>();        // salida: { dni, password }

  protected alEnviar(): void {
    ...
    this.enviar.emit({ dni: normalizarDni(dni), password });
  }
}
```

> **Para ser honestos en el examen**
>
> No usamos `@Input()` ni `@Output()` : es la forma anterior y sigue funcionando. Si te piden escribirlo así: `@Input() cargando = false;` y `@Output() enviar = new EventEmitter<CredencialesLogin>();` . La diferencia: `input()` es un *signal*, así que se lee `cargando()` y se puede usar dentro de un `computed` .

Otra forma: proyección de contenido. `EstructuraPanel` (`shared/ui/`) dibuja el menú, el encabezado y la campana de avisos, y en su `<ng-content />` pone lo que cada pantalla escribe entre `<app-estructura-panel>` y su cierre.

1.6 Directivas EN EL PROYECTO

Concepto. Una directiva le agrega comportamiento a un elemento del template. Las **de atributo** cambian su aspecto o comportamiento (`[class]` , `routerLink`). Las **estructurales** agregan o quitan elementos del DOM (`*ngIf` , `*ngFor` ; hoy reemplazadas por `@if` y `@for`).

En el proyecto. `*siRol` es una directiva estructural propia: muestra un bloque solo si la sesión tiene alguno de los roles indicados.

```ts
// shared/directivas/si-rol/si-rol.ts
@Directive({ selector: '[siRol]' })
export class SiRol {
  private readonly auth = inject(AuthService);
  private readonly plantilla = inject<TemplateRef<unknown>>(TemplateRef);  // el bloque marcado
  private readonly contenedor = inject(ViewContainerRef);                  // dónde se dibuja
  readonly siRol = input.required<string | readonly string[]>();
  private vista: EmbeddedViewRef<unknown> | null = null;

  constructor() {
    effect(() => {
      const roles = this.siRol();
      const permitidos = typeof roles === 'string' ? [roles] : roles;
      const mostrar = tieneAlgunRol(this.auth.sesion(), permitidos);
      if (mostrar && this.vista === null) {
        this.vista = this.contenedor.createEmbeddedView(this.plantilla);
      } else if (!mostrar && this.vista !== null) {
        this.contenedor.clear();
        this.vista = null;
      }
    });
  }
}
```

> **`*siRol` no es seguridad**
>
> Oculta botones en el navegador, nada más. Si alguien llama igual a la API, el backend responde 403 (ver la guía del backend, 6.5).

3. Eje 2 – Routing y formularios

2.1 Rutas y router-outlet EN EL PROYECTO

Concepto. El router asocia cada URL con un componente sin recargar la página. Las rutas son un arreglo de tipo `Routes`, se registran con `provideRouter(routes)` y `<router-outlet>` marca dónde se dibuja el componente de la ruta activa. Lo que está fuera del outlet no se destruye al navegar.

En el proyecto. `app.routes.ts` tiene 36 rutas (en el material la constante se llama `routeConfig`; acá, `routes`). `provideRouter(routes)` va en `app.config.ts`, que `main.ts` le pasa a `bootstrapApplication`.

```
// app.routes.ts (resumido)
export const routes: Routes = [
  {
    path: 'login',
    title: 'Iniciar sesión · ISCGB',
    loadComponent: () => import('./features/auth/login/login').then((m) => m.Login),
  },
  {
    path: 'director/panel',
    title: 'Panel del Director · ISCGB',
    canActivate: [authGuard, roleGuard(ROLES.director)],
    loadComponent: () => …,
  },
  ...
  { path: '', pathMatch: 'full', redirectTo: 'login' },   // la raíz va al login
  { path: '**', title: 'Página no encontrada · ISCGB', loadComponent: () => … },  // siempre la
última
];
```

Propiedad	Para qué	Ejemplo en el proyecto
`path`	La URL, sin barra inicial.	`'docente/entrega-programa'`
`loadComponent`	Qué componente cargar, de forma diferida.	`() => import('…').then((m) => m.PanelAlumno)`
`title`	Título de la pestaña del navegador.	`'Mi legajo · ISCGB'`
`canActivate`	Guards para entrar.	`[authGuard, roleGuard(ROLES.docente)]`
`canDeactivate`	Guard para salir.	`[confirmarSalidaGuard]` en `docente/entrega-programa`
`data`	Datos fijos para el componente.	`{ variante: 'regular' }`: dos rutas, una sola pantalla de certificado.
`redirectTo` + `pathMatch`	Redirigir. `'full'` exige que la URL entera coincida.	`''` → `'login'`

> **Pregunta posible: ¿qué pasa si alguien escribe una URL que no existe?**
> Sin una ruta comodín, la navegación falla ("Cannot match any routes") y la vista no cambia. Nosotros tenemos `path: '**'` al final, que muestra `NoEncontrado` con un botón para volver al panel propio. Va última porque el router usa la **primera** ruta que coincide.

2.2 Navegación: routerLink y Router EN EL PROYECTO

Concepto. Con `href` el navegador vuelve a pedir la página: se recarga la app entera y se pierde el estado. `routerLink` intercepta el clic y cambia la vista sin recargar (usa la History API del navegador). Con corchetes, `[routerLink]`, el valor es una expresión (una variable o un arreglo); sin corchetes, un texto fijo. Desde el código se navega con el servicio `Router`.

Forma	Ejemplo en el proyecto
`routerLink` fijo	`routerLink="/recuperar-contrasena"` en `formulario-login.html`
`[routerLink]` dinámico + `routerLinkActive`	`[routerLink]="enlace.url"` en el menú de `estructura-panel.html`; `routerLinkActive` resalta la página actual.
`router.navigate([…])`	`this.router.navigate([destinoSegunRoles(sesion)])` después del login.
`router.createUrlTree([…])`	En los guards: devuelve una redirección en lugar de navegar (ver 2.5).

2.3 Parámetros de ruta y query params EN EL PROYECTO

Concepto. Los dos puntos en el `path` (`:idUsuario`) marcan un **parámetro de ruta**: un segmento variable con nombre. Se lee inyectando `ActivatedRoute`. El *snapshot* es una foto de la ruta al crear el componente; si el valor puede cambiar sin destruirlo, se usa el observable (`paramMap` o `queryParamMap`). Los parámetros **siempre llegan como texto**. Los **query params** son lo que va después del `?`.

```
// app.routes.ts
{ path: 'director/usuarios/:idUsuario/editar', … }

// features/director/editar-usuario/editar-usuario.ts
private readonly route = inject(ActivatedRoute);
private readonly idUsuario = Number(this.route.snapshot.paramMap.get('idUsuario'));  // '7' → 7

// features/auth/login/login.ts — ?sesionVencida=1 leído como signal
protected readonly sesionVencida = toSignal(
  this.route.queryParamMap.pipe(map((params) => params.get(PARAMETRO_SESION_VENCIDA) === '1')),
  { initialValue: false },
);
```

Query param	Quién lo pone	Qué hace la pantalla
`?sesionVencida=1`	`authGuard` y `sesionInterceptor`	El login muestra "Tu sesión venció".
`?accesoDenegado=1`	`roleGuard` y `sesionInterceptor` (403)	El panel propio muestra el cartel de acceso denegado.
`?tipo=`	La tarjeta "Documentación por entregar" y la campana de avisos	`subir-documento` preselecciona ese tipo de documento.

2.4 Carga diferida (lazy loading) EN EL PROYECTO

Concepto. Con `component:` el componente entra en el paquete inicial. Con `loadComponent` y un `import()` dinámico, el empaquetador lo separa en su propio archivo JS (*chunk*) y lo descarga recién cuando alguien entra a esa ruta. El arranque es más liviano aunque haya decenas de pantallas.

En el proyecto. Todas las rutas que muestran una pantalla usan `loadComponent`. Para verlo: `F12 → Network → JS` y navegar a otra pantalla; baja un archivo nuevo.

2.5 Guards EN EL PROYECTO

Concepto. Un guard decide si se puede **entrar** a una ruta (`CanActivate`) o **salir** de ella (`CanDeactivate`). Devuelve `true` o una redirección (`UrlTree`). En el Angular actual se escriben como funciones (`CanActivateFn`); antes eran clases que implementaban `CanActivate` .

Guard	Qué pregunta	Si no pasa
`authGuard`	¿Hay sesión y el token sigue vigente?	Va a `/login` , o a `/login?sesionVencida=1` si venció.
`roleGuard(...roles)`	¿La sesión tiene alguno de esos roles?	Va al panel propio con `?accesoDenegado=1` .
`confirmarSalidaGuard`	¿Hay un programa sin enviar?	Pregunta antes de salir de `docente/entrega-programa` .

```
// core/auth/auth.guard.ts
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  const sesion = auth.sesion();
  if (sesion === null) {
    return router.createUrlTree(['/login']);
  }
  if (!sesionVigente(sesion)) {
    auth.cerrarSesion();
    return router.createUrlTree(['/login'], { queryParams: { [PARAMETRO_SESION_VENCIDA]: 1 } });
  }
  return true;
};

// core/auth/role.guard.ts — una "fábrica": recibe los roles y devuelve el guard
export function roleGuard(...permitidos: readonly string[]): CanActivateFn {
  return () => {
    ...
    if (tieneAlgunRol(sesion, permitidos)) {
      return true;
    }
    return router.createUrlTree([destinoSegunRoles(sesion)], {
      queryParams: { [PARAMETRO_ACCESO_DENEGADO]: 1 },
    });
  };
}
```

Un caso que vale la pena explicar

`roleGuard` es una **fábrica**: una función que recibe los roles permitidos y devuelve el guard, que los "recuerda" (*closure*). Por eso en las rutas se escribe `roleGuard(ROLES.director, ROLES.secretario)`. Además devuelve un `UrlTree` en lugar de llamar a `router.navigate`: así Angular cancela la navegación y redirige en un solo paso.

Un guard no es seguridad

Corre en el navegador del usuario: se saltea con DevTools o llamando directo a la API con Postman. El guard mejora la experiencia; la seguridad está en el backend, con `[Authorize(Roles = …)]` y el control por dueño del dato (ver la guía del backend, 6.5).

2.6 Formularios reactivos y validaciones EN EL PROYECTO

Concepto. Angular tiene dos formas de armar formularios. En los **template-driven** (`ngModel`) el modelo vive en el HTML. En los **reactivos** el modelo vive en la clase: un `FormGroup` que agrupa `FormControl`, y el HTML se conecta con `[formGroup]` y `formControlName`. Los reactivos se tipan, se validan y se prueban más fácil. Validadores incluidos: `Validators.required`, `email`, `minLength`, `pattern`… y se pueden escribir propios.

En el proyecto. Por regla del equipo, todos los formularios son reactivos y tipados, armados con `FormBuilder.nonNullable`: cada control es `string` y nunca `null`, y al resetear vuelve a `''`.

```
// formulario-login.ts
protected readonly formulario = this.fb.nonNullable.group({
  dni: ['', [Validators.required]],
  password: ['', [Validators.required]],
});

protected alEnviar(): void {
  this.seIntentoEnviar.set(true);              // recién ahora se muestran los errores
  if (this.formulario.invalid || this.cargando()) {
    return;
  }
  const { dni, password } = this.formulario.getRawValue();
  this.enviar.emit({ dni: normalizarDni(dni), password });   // "12.345.678" → "12345678"
}

<!-- formulario-login.html (resumido, sin estilos) -->
<form [formGroup]="formulario" (ngSubmit)="alEnviar()" novalidate>
  <input id="dni" type="text" formControlName="dni" [attr.aria-invalid]="errorDni !== null" />
  ...
</form>
```

Validación de formato en el front	Dónde
Campos obligatorios	`Validators.required` en el login.
DNI de 7 u 8 números	`esDniValido` en `core/auth/dni.ts`.
Archivo PDF de hasta 10 MB	`validarArchivoPdf` en `core/comun/archivos.ts`, que usa `zona-archivo`.
Foto de perfil de hasta 2 MB	`core/comun/archivos.ts`.

Para ser honestos en el examen

Las validaciones del front son **comodidad**: avisan antes de mandar. El backend vuelve a validar todo, incluido el contenido real del archivo (ver la guía del backend, 5.5 y 5.6). Si el back rechaza con 400, `mensajeDelServidor` (`core/comun/error-api.ts`) muestra el mensaje que mandó, incluso los errores por campo de `[ApiController]` .

4. Eje 3 – Servicios, comunicación con la API y estado

3.1 Servicios e inyección de dependencias EN EL PROYECTO

Concepto. Un servicio es una clase con lógica que no pertenece a la pantalla: llamar a la API, guardar estado compartido, hacer cálculos. Con **inyección de dependencias** la clase no crea lo que necesita con `new`: lo **pide**, y Angular se lo entrega. Ventajas: bajo acoplamiento, se puede cambiar una implementación por otra (por ejemplo, un mock) y es más fácil probar.

En el proyecto. Hay dos formas de registrar un servicio y dos de pedirlo:

```
// Registrar 1: el servicio se registra solo; una instancia para toda la app (Singleton)
@Injectable({ providedIn: 'root' })
export class TemaService { ... }

// Registrar 2: en app.config.ts, eligiendo la implementación de una clase abstracta
{ provide: AuthService, useClass: AuthHttpService },

// Pedir 1: con inject() en un campo (lo que usamos)
private readonly auth = inject(AuthService);

// Pedir 2: por constructor (como en el PPT de Login)
constructor(private http: HttpClient) {}
```

La misma idea que en el backend

	Angular	ASP.NET Core
Registrar una implementación	`{ provide: AuthService, useClass: AuthHttpService }`	`AddScoped<IEmailService, EmailService>()`
Una instancia para toda la app	`providedIn: 'root'` (y lo registrado en `app.config.ts`)	`AddSingleton`
Pedir la dependencia	`inject(AuthService)` o el constructor	Parámetro del constructor

> **Detalle del proyecto**
>
> `inject()` solo funciona en **contexto de inyección**: al inicializar los campos, en el constructor y en guards e interceptores funcionales. Por eso `sesionInterceptor` lo llama al principio y no dentro de un `catchError`.

3.2 Servicios con mock: clase abstracta y useClass EN EL PROYECTO

Concepto. Mockear es reemplazar la fuente real por una **falsa y controlada** que responde igual. Sirve para avanzar aunque el backend no tenga el endpoint, probar sin red y tener resultados predecibles. La clave es programar contra una abstracción y elegir la implementación en un solo lugar (patrones Strategy e inversión de dependencias).

En el proyecto. Cada módulo de `core/` tiene un trío: `x.service.ts` (la clase abstracta), `x-http.service.ts` (la API real) y, en la mayoría, `x-mock.service.ts` (datos en memoria). Hoy los 14 servicios registrados en `app.config.ts` usan la versión HTTP, y 11 tienen además su mock.

```
// core/contratos/contratos.service.ts — el contrato
export abstract class ContratosService {
  abstract cargar(contrato: NuevoContrato): Observable<string>;
  abstract listarDeAlumno(idUsuario: number): Observable<ContratoAlumno[]>;
  abstract misContratos(): Observable<ContratoAlumno[]>;
  abstract descargar(idContrato: number): Observable<Blob>;
}

// core/contratos/contratos-mock.service.ts — datos en memoria
@Injectable()
export class ContratosMockService extends ContratosService {
  private guardados: ContratoGuardado[] = [];
  ...
  misContratos(): Observable<ContratoAlumno[]> {
    const idUsuario = this.auth.sesion()?.idUsuario;
    return of(idUsuario === undefined ? [] : this.de(idUsuario)).pipe(delay(DEMORA_SIMULADA_MS));
  }
}

// app.config.ts — se elige cuál usar
{ provide: ContratosService, useClass: ContratosHttpService },   // o ContratosMockService
```

Dónde nos sirvió

- **Contratos de alumnos** (SCRUM-227) se armó primero con `ContratosMockService`, cuando el backend todavía no tenía `api/ContratosAlumno`. Cuando estuvieron los endpoints, se cambió una línea de `app.config.ts`; las pantallas no se tocaron.
- **Con el backend caído** se trabaja igual: `AuthMockService` y `LegajoMockService` en lugar de los HTTP. Los usuarios de prueba (uno por rol) están en `core/auth/usuarios-de-prueba.ts`.
- **En los tests** pasa lo mismo: `role.guard.spec.ts` inyecta un `AuthService` falso con `{ provide: AuthService, useClass: AuthDePrueba }`, y el guard no nota la diferencia.

3.3 HttpClient y Observables EN EL PROYECTO

Concepto. `HttpClient` es el servicio de Angular para pedirle datos a una API; se habilita con `provideHttpClient()`. Cada método (`get`, `post`, `put`, `delete`) devuelve un **Observable**: un flujo de valores en el tiempo. El pedido **no sale hasta que alguien se suscribe** (`subscribe`). Con `pipe` se encadenan operadores de RxJS que transforman el resultado o manejan los errores.

Método	Para qué	Llamadas	Ejemplo en el proyecto
`GET`	Leer.	30	`RUTAS_API.misContratos`
`POST`	Crear o ejecutar una acción.	17	`RUTAS_API.login`; cargar un contrato con `FormData`
`PUT`	Actualizar.	12	`RUTAS_API.modificarUsuario(id)`; la baja lógica, `RUTAS_API.bajaUsuario(id)`
`DELETE`	Borrar.	1	`RUTAS_API.eliminarExamen(idExamen)` en el calendario

Cantidades contadas en los 14 servicios `*-http.service.ts` de `core/`.

```
// core/auth/auth-http.service.ts — el servicio arma el pedido y lo adapta
iniciarSesion(credenciales: CredencialesLogin): Observable<Sesion> {
  return this.http.post<RespuestaLogin>(RUTAS_API.login, credenciales).pipe(
    map((respuesta) => this.aSesion(respuesta)),       // JSON de la API → Sesion del front
    map((sesion) => {
      this.guardar(sesion);                            // signal + sessionStorage
      return sesion;
    }),
    catchError((error: HttpErrorResponse) => {
      const mensaje = mensajeDeAcceso(
        error,
        { 401: MENSAJE_CREDENCIALES_INVALIDAS },
        MENSAJE_SIN_CONEXION,
      );
      return throwError(() => new Error(mensaje));
    }),
  );
}

// features/auth/login/login.ts — recién acá sale el pedido
this.auth.iniciarSesion(credenciales).subscribe({
  next: (sesion) => {
    this.cargando.set(false);
    this.router.navigate([destinoSegunRoles(sesion)]);
  },
  error: (fallo: Error) => {
    this.cargando.set(false);
    this.error.set(fallo.message);
  },
});
```

Operador de RxJS	Qué hace	Dónde
`map`	Transforma cada valor.	`aSesion` en `auth-http.service.ts`.
`catchError` + `throwError` / `of`	Atrapa el error y lo traduce o lo reemplaza.	`auth-http.service.ts`; un 404 pasa a lista vacía en `legajo-http.service.ts`.
`finalize`	Corre al terminar, bien o mal.	`cargaInterceptor`.
`forkJoin`	Espera varios pedidos y entrega todo junto.	`cargarLegajoPropio` (documentos y requeridos).
`Subject` + `startWith` + `switchMap`	Reintentar: cada `next()` cancela el pedido anterior y lanza uno nuevo.	`panel-alumno.ts`.
`toSignal`	Convierte un Observable en signal.	`login.ts`, `panel-alumno.ts`.

Archivos: subir y bajar

Para subir un PDF se manda un `FormData` (*multipart/form-data*), que el backend recibe con `[FromForm]`. Para bajar uno se pide con `responseType: 'blob'`. Los dos casos están en `ContratosHttpService`:

```
const cuerpo = new FormData();
cuerpo.append('idUsuario', String(contrato.idUsuario));
cuerpo.append('nombre', contrato.nombre);
cuerpo.append('archivo', contrato.archivo);
return this.http.post<RespuestaApi>(RUTAS_API.cargarContratoAlumno, cuerpo) ...

return this.http.get(RUTAS_API.pdfContratoAlumno(idContrato), { responseType: 'blob' }) ...
```

3.4 Interceptores EN EL PROYECTO

Concepto. Un interceptor se mete en el medio de **todos** los pedidos HTTP: puede modificar el pedido antes de que salga y reaccionar a la respuesta. Los pedidos son inmutables, así que para agregar un encabezado hay que **clonarlos**. Se registran en orden con `withInterceptors([…])`. Es el patrón Chain of Responsibility, y la misma idea que el *pipeline* de middleware del backend.

Interceptor	Qué hace
`tokenInterceptor`	Si la URL es de nuestra API y hay sesión, clona el pedido y agrega `Authorization: Bearer <token>`. Va primero para que el pedido salga con el token.
`sesionInterceptor`	Mira la respuesta. Con 401 cierra la sesión y lleva a `/login?sesionVencida=1`; con 403 lleva al panel propio con `?accesoDenegado=1`. Ignora la URL del login.
`cargaInterceptor`	Muestra el logo de espera global y lo oculta con `finalize`. Un pedido puede quedar afuera marcándolo con `SIN_CARGA_GLOBAL`.

```
// core/auth/token.interceptor.ts
export const tokenInterceptor: HttpInterceptorFn = (peticion, siguiente) => {
  if (!peticion.url.startsWith(URL_BASE_API)) {
    return siguiente(peticion);                       // nunca le mandamos el token a otro
servidor
  }
  const token = inject(AuthService).sesion()?.token;
  if (token === undefined) {
    return siguiente(peticion);                       // sin sesión (por ejemplo, el login)
  }
  return siguiente(
    peticion.clone({ setHeaders: { Authorization: `Bearer ${token}` } }),
  );
};
```

Pregunta posible: ¿por qué `sesionInterceptor` ignora la URL del login?

Porque en el login un 401 significa "DNI o contraseña incorrectos", no "se venció la sesión". Ese error lo traduce `AuthHttpService` en un mensaje para la pantalla.

3.5 Signals: signal, computed y effect EN EL PROYECTO

Concepto. Un *signal* es un envoltorio reactivo alrededor de un valor: avisa a quien lo lee cuando cambia. En el modelo tradicional, Zone.js revisaba todo el árbol de componentes ante cualquier evento; con signals la actualización es **granular**: solo se redibuja lo que leyó ese valor.

Pilar	Qué es	En el proyecto
`signal`	Estado escribible. Se lee llamándolo, `x()`, y se cambia con `.set()` o `.update()`.	`preferencia` en `TemaService`; `cargando` y `error` en `Login`.
`computed`	Estado derivado, de solo lectura. Perezoso (calcula al leerse) y memorizado (guarda el resultado hasta que cambie algo).	`tema` en `TemaService`; `estaAutenticado` en `AuthHttpService`.
`effect`	Efecto hacia afuera (DOM, storage, consola). No es para cambiar estado.	`TemaService` pone `data-tema` en `<html>`; `*siRol` crea o borra su bloque.

```
// core/tema/tema.service.ts (resumido) — el modo claro / oscuro
@Injectable({ providedIn: 'root' })
export class TemaService {
  // lo que eligió la persona: 'claro' | 'oscuro' | 'sistema'
  readonly preferencia = signal<PreferenciaTema>(leerPreferenciaTema(this.leerGuardada()));
  // lo que dice el sistema operativo
  private readonly sistemaOscuro = signal(this.consulta?.matches ?? false);
  readonly tema = computed<TemaAplicado>(() =>
    resolverTema(this.preferencia(), this.sistemaOscuro()),
  );

  constructor() {
    ...
    effect(() => this.aplicar(this.tema()));        // escribe data-tema="oscuro" en <html>
  }

  establecer(preferencia: PreferenciaTema): void {
    this.preferencia.set(preferencia);
    this.guardar(preferencia);                       // localStorage
  }
}
```

Otras piezas que usamos	Para qué
`toSignal`	Puente de RxJS a signals: el query param del login, el legajo del panel del alumno.
`linkedSignal`	Un valor derivado que la persona puede pisar: en `subir-documento`, el tipo arranca con el `?tipo=` de la URL y se cambia en el select.
`input()` / `output()`	Entradas y salidas de un componente (ver 1.5).
`untracked`	Leer un signal dentro de un `effect` sin volverlo dependencia (`CargaService`).

Detalle del proyecto: zoneless y OnPush

La app es *zoneless*: no hay `zone.js` en `package.json`. Angular se entera de los cambios porque los signals avisan. Además, por regla del equipo, los componentes usan `ChangeDetectionStrategy.OnPush`: un componente se redibuja solo cuando cambia un `input()`, un signal que lee su template o hay un evento en él.

Signals y RxJS no compiten

Signals para el estado que se dibuja; RxJS para lo asincrónico (HTTP, eventos en el tiempo). `toSignal` es el puente entre los dos.

3.6 Sesión y JWT del lado del front EN EL PROYECTO

Concepto. Después del login el front guarda el token y lo manda en cada pedido. El front puede **leer** el JWT, porque está en Base64URL y no cifrado, pero **no puede verificarlo**: la firma la valida el backend con su clave secreta. Lo que el front lee sirve para decidir qué mostrar, nunca qué permitir.

En el proyecto, paso a paso:

1. `AuthHttpService.iniciarSesion` recibe `{ token, idUsuario, usuario, dni, email, roles, foto }`.
2. `aSesion` arma la `Sesion` (patrón Adapter). Del token solo lee el vencimiento, `exp`, con `leerPayloadJwt`. Los roles salen del cuerpo: `roles: [{ idRol, nombreRol }]`.
3. La guarda en un signal y en `sessionStorage`, con la clave `iscgb.sesion`.
4. `tokenInterceptor` la agrega en cada pedido; `authGuard` controla que siga vigente al navegar.
5. Al recargar con F5, `leerSesionGuardada` la recupera; si venció o está dañada, la descarta.
6. `cerrarSesion()` borra el signal y el storage.

```
// core/auth/jwt.ts — leer el contenido del token (no verificarlo)
export function leerPayloadJwt(token: string): JwtPayload | null {
  const partes = token.split('.');                    // HEADER.PAYLOAD.FIRMA
  if (partes.length !== 3) {
    return null;
  }
  try {
    const contenido = partes[1].replace(/-/g, '+').replace(/_/g, '/');   // Base64URL → Base64
    const json = decodeURIComponent(
      atob(contenido)
        .split('')
        .map((caracter) => '%' + caracter.charCodeAt(0).toString(16).padStart(2, '0'))
        .join(''),
    );
    return JSON.parse(json) as JwtPayload;
  } catch {
    return null;
  }
}
```

Decisión	Por qué
`sessionStorage` y no `localStorage` para la sesión	La sesión muere al cerrar la pestaña: importante en las computadoras compartidas de Secretaría.
El tema sí va en `localStorage`	Es una preferencia, no una credencial.

Decisión	Por qué
Mismo mensaje para DNI inexistente y contraseña incorrecta	No revelar qué DNI están registrados (enumeración de usuarios).
Mensajes propios para el estado 0 y el 429	0 = no hubo conexión con el servidor; 429 = el backend frenó demasiados intentos seguidos (*rate limiting*).

Lo que hay y lo que no hay en el proyecto

- **Riesgo conocido:** cualquier script que corra en la página puede leer `sessionStorage` (XSS). Lo mitigan que Angular escapa todo lo que se interpola, que el token dura 2 horas y que muere al cerrar la pestaña.
- **Cookie `HttpOnly`** NO ESTÁ: sería lo más seguro (JavaScript no la puede leer), pero la tiene que emitir el backend.
- **Archivos de entorno** NO ESTÁ: la URL de la API está fija en `core/configuracion/api.ts` (`http://localhost:5231`). Como está en un solo lugar, para el despliegue alcanza con `fileReplacements` de `angular.json` o una configuración leída al arrancar.

5. Puntos de contacto con el backend (ejes 4 a 6)

Estos temas son del backend, pero el examen puede relacionarlos con el frontend. Los números son los de la guía del backend.

Tema del backend	Relación con el frontend
Controladores y rutas (4.1)	Cada URL de la API está una sola vez en `core/configuracion/api.ts` (`RUTAS_API`) y los servicios HTTP la usan desde ahí. Si cambia una ruta del backend, se cambia en un solo lugar.
DTO (4.2)	Las interfaces de TypeScript son su espejo: `CredencialesLogin` tiene la forma de `LoginRequestDto` y `RespuestaLogin` describe lo que devuelve el login. Si cambia un DTO, hay que actualizar la interfaz.
Datos mockeados (4.3)	Los `*-mock.service.ts` permiten trabajar sin backend; se elige con `useClass` en `app.config.ts` (ver 3.2).
Inyección de dependencias (4.5)	Misma idea en los dos lados: `inject()` + `useClass` frente a constructor + `AddScoped`. `providedIn: 'root'` es un Singleton.
Métodos HTTP (5.2)	`HttpClient.get/post/put/delete`: 30 GET, 17 POST, 12 PUT y 1 DELETE en los servicios del front. La baja de usuario también es un PUT (baja lógica).
Códigos de estado (5.3)	401 y 403 los maneja `sesionInterceptor`; un 400 muestra el mensaje del servidor (`mensajeDelServidor`); un 404 se convierte en lista vacía en los legajos; el 429 tiene mensaje propio en el login.
Model binding (5.4)	`[FromBody]` recibe el JSON de `http.post(url, objeto)`; `[FromForm]` recibe el `FormData` de las subidas de PDF (mismos nombres: `idUsuario`, `nombre`, `archivo`).
Validaciones (5.5 y 5.6)	El front valida el formato por comodidad (DNI, PDF de hasta 10 MB); el backend valida todo de nuevo, incluido el contenido real del archivo.
JWT (6.4)	El front lo guarda y lo manda en `Authorization: Bearer`; solo lee `exp`. La firma la valida el backend.
Autenticación y autorización (6.5)	`authGuard`, `roleGuard` y `*siRol` ordenan la navegación; la protección real es `[Authorize(Roles)]` y el control por dueño del dato.
CORS (6.6)	El backend solo acepta llamadas desde el origen configurado del front (`Frontend:Url`). Si el front corre en otro origen, el navegador bloquea las respuestas.

6. Preguntas tipo examen

¿Qué es una SPA y qué ventaja tiene?

Una aplicación que carga una sola página y cambia la vista sin recargar. Baja la app una vez y después solo pide datos en JSON: menos tráfico y navegación más rápida.

¿Por qué Angular es un framework y React una librería?

Angular trae todo: router, formularios, HttpClient, inyección de dependencias, CLI y TypeScript obligatorio, con una estructura fija. React resuelve la vista y el equipo elige el resto.

¿Qué pasa con los tipos de TypeScript al compilar?

Se revisan y después se borran: el navegador recibe JavaScript. Por eso TypeScript no valida datos en ejecución, y por eso está en `devDependencies`.

¿Diferencia entre interface y clase? ¿Por qué `AuthService` es abstracta?

La interface es solo un contrato y no genera código; la clase existe en ejecución. `AuthService` es una clase abstracta porque Angular la usa como llave de la inyección de dependencias.

¿Cuáles son las partes de un componente? Dé un ejemplo.

Clase TypeScript (comportamiento), template HTML (estructura) y estilos (diseño), unidos por el decorador `@Component`. Ej.: `Login`, con `login.ts` y `login.html`.

¿Qué tipos de data binding hay?

Interpolación `{{ }}` y property binding `[ ]` (clase → vista), event binding `( )` (vista → clase) y el enlace de formularios (`[formGroup]` + `formControlName`).

¿Cómo se comunican un componente padre y uno hijo?

El padre pasa datos por `input()` (`[cargando]="cargando()"`) y el hijo avisa con `output()` (`(enviar)="iniciarSesion($event)"`). Antes se usaban `@Input()` y `@Output()`.

¿Qué es una directiva estructural? Dé un ejemplo del proyecto.

Una directiva que agrega o quita elementos del DOM. `*siRol` dibuja un bloque solo si la sesión tiene alguno de los roles, con `TemplateRef` y `ViewContainerRef`.

¿Qué hace `<router-outlet>`?

Marca dónde se dibuja el componente de la ruta activa. Lo que está afuera persiste al navegar; en ISCGB, el cargador global.

¿Diferencia entre `href` y `routerLink`? ¿Y con corchetes?

`href` recarga la app entera; `routerLink` cambia la vista sin recargar. `[routerLink]` evalúa una expresión (una variable o un arreglo), sin corchetes es texto fijo.

¿Cómo se lee un parámetro de ruta? ¿Por qué `Number()`?

Inyectando `ActivatedRoute`: `this.route.snapshot.paramMap.get('idUsuario')`. Los parámetros llegan siempre como texto, así que hay que convertirlos.

¿Qué es lazy loading y cómo lo usan?

Cargar el código de una pantalla recién cuando se entra. Cada ruta usa `loadComponent: () => import(…)`, y el empaquetador genera un archivo JS por pantalla.

¿Qué es un guard? ¿Alcanza para proteger una ruta?

Una función que decide si se puede entrar a una ruta o salir de ella. No alcanza: corre en el navegador y se puede saltear. La seguridad real está en el backend con `[Authorize]`.

¿Formularios reactivos o template-driven?

En los reactivos el modelo vive en la clase (`FormGroup` , `FormControl`) y se tipa y valida más fácil. Usamos reactivos con `FormBuilder.nonNullable` .

¿Qué es un servicio y qué es la inyección de dependencias?

Un servicio es una clase con lógica que no es de la pantalla. Con inyección de dependencias la clase no crea sus dependencias: las pide con `inject()` o por constructor y Angular se las entrega.

¿Qué significa `providedIn: 'root'` ?

Que el servicio se registra en el inyector raíz: hay una sola instancia para toda la aplicación (Singleton). Ej.: `TemaService` y `CargaService` .

¿Qué es mockear y cómo lo hicieron?

Reemplazar la fuente real por una falsa que responde igual. Cada servicio tiene una clase abstracta, una versión HTTP y una mock; en `app.config.ts` se elige cuál con `useClass` .

¿Qué devuelve `HttpClient` ? ¿Cuándo sale el pedido?

Un Observable. El pedido sale recién cuando alguien se suscribe con `subscribe` (o con `toSignal`).

¿Qué es un interceptor? Nombre los del proyecto y su orden.

Una función que se mete en todos los pedidos HTTP. `tokenInterceptor` (agrega el Bearer), `sesionInterceptor` (401 y 403) y `cargaInterceptor` (logo de espera), en ese orden.

¿Diferencia entre `signal` , `computed` y `effect` ?

`signal` guarda un valor que se puede cambiar; `computed` deriva un valor de otros signals y es de solo lectura; `effect` ejecuta algo hacia afuera cuando cambian los signals que lee. Ej.: `preferencia` , `tema` y el `effect` que pinta `data-tema` .

¿Dónde guardan la sesión y por qué ahí?

En un signal y en `sessionStorage` (`iscgb.sesion`), para que la sesión muera al cerrar la pestaña.

¿El front verifica el JWT?

No. Lo lee (Base64URL) solo para saber cuándo vence. La firma la verifica el backend.

Explique el flujo de inicio de sesión del lado del front.

`FormularioLogin` valida y emite `{ dni, password }` → `Login` llama a `AuthService.iniciarSesion` → `AuthHttpService` hace el POST → los interceptores lo acompañan → `aSesion` arma la `Sesion` → se guarda en signal y `sessionStorage` → `router.navigate` al panel del rol → `authGuard` y `roleGuard` dejan entrar.

7. Glosario

Término	Significado
SPA	Aplicación de una sola página que cambia la vista sin recargar.
Transpilación	Convertir TypeScript en JavaScript que entiende el navegador.
Componente	Bloque de la interfaz: clase, template y estilos.
Standalone	Componente que declara sus propios `imports`, sin `NgModule`.
Decorador	Anotación (`@Component`, `@Injectable`) que le agrega metadata a una clase.
Data binding	Enlace entre la clase y el template: interpolación, property y event binding.
Directiva	Comportamiento agregado a un elemento del template; las estructurales agregan o quitan elementos.
Router	Servicio que asocia URLs con componentes y navega sin recargar.
Lazy loading	Cargar el código de una pantalla recién cuando se entra a su ruta.
Guard	Función que permite o bloquea entrar a una ruta (o salir de ella).
UrlTree	Una URL armada por el router; si un guard la devuelve, la navegación se redirige ahí.
Formulario reactivo	Formulario cuyo modelo (`FormGroup`, `FormControl`) vive en la clase.
Servicio	Clase con lógica que no pertenece a la pantalla: API, estado compartido, cálculos.
Inyección de dependencias	La clase pide lo que necesita y el framework se lo entrega.
Provider	Registro que le dice a Angular qué entregar cuando alguien pide algo (`provide` + `useClass`).
Mock	Implementación falsa y controlada que responde igual que la real.
Observable	Flujo de valores en el tiempo (RxJS); lo devuelve `HttpClient`.
Interceptor	Función que se mete en todos los pedidos HTTP antes y después de la API.
Signal	Valor reactivo que avisa a quien lo lee cuando cambia.
Zoneless	Angular sin Zone.js: se entera de los cambios por los signals.
OnPush	Estrategia que redibuja un componente solo cuando cambian sus entradas, sus signals o hay un evento.
Contenedor / presentacional	El contenedor maneja datos y servicios; el presentacional solo recibe y emite.
Adapter	Función que convierte la respuesta de la API en el modelo del front (`aSesion`).